

EXPERIMENT-9: Custom Node.js server and core modules.

Create a custom server using the Node.js http module and explore Node.js modules like os, path, and events.

Objective : Build a basic server and practice common Node.js core modules.

Tools : Node.js, http, os, path, events

Outcome : A working custom Node.js server plus exploration of key core modules.

Answer:

Use http.createServer to handle requests, inspect request details, and use os/path utilities for environment and path handling.

📁 Project Structure

NodeCoreModulesServer/

```
├── server.js      -> http.createServer + routing
├── utils/
|   ├── logger.js  -> events module: request logger (EventEmitter)
|   ├── osInfo.js  -> os module: system info
|   ├── pathInfo.js -> path module: path parsing utilities
|   └── orderEvents.js -> events module: multi-listener + once() demo
├── data/
|   └── books.json  -> sample data read via fs + path
└── package.json
```

Prerequisites

Just Node.js (any recent LTS, e.g. v18+). No npm packages needed — everything here is a Node core module.

Run

```
```bash
```

```
cd NodeCoreModulesServer
```

```
node server.js
```

```
```
```

or

```
```bash
```

```
npm start
```

You should see:

Server running at http://localhost:3000/

Open `http://localhost:3000/` in a browser, or use `curl`.

**File Name:** books.json

Code:

```
[
 { "id": 1, "title": "Node.js Essentials", "author": "Sarah Mitchell", "price": 299.00 },
 { "id": 2, "title": "Mastering XML Schemas", "author": "Rajesh Kumar", "price": 450.50 },
 { "id": 3, "title": "Web Development with Express", "author": "Emily Chen", "price": 375.00 }
]
```

**File Name :** logger.js

Code:

```
// utils/logger.js
// Demonstrates the "events" core module: a custom EventEmitter
// subclass used to log every incoming HTTP request.
```

```
const EventEmitter = require("events");
class AppLogger extends EventEmitter {}
const logger = new AppLogger();
// Listener: prints a formatted log line for every 'request' event
logger.on("request", (data) => {
 console.log(`[${data.time}] ${data.method} ${data.url}`);
});
module.exports = logger;
```

**File Name :** orderEvents.js

```
// utils/orderEvents.js
// A second, more elaborate "events" module demo: multiple listeners
// reacting to the same event, plus emitter.once() usage.
const EventEmitter = require("events");
function demonstrateEvents() {
 const emitter = new EventEmitter();
 const log = [];

 // Listener 1: order received
 emitter.on("bookOrdered", (book) => {
```

```

 log.push(`Order received for "${book.title}" (${book.price})`);
 });
 // Listener 2: confirmation email (runs on every emit)
 emitter.on("bookOrdered", (book) => {
 log.push(`Confirmation email queued for "${book.title}"`);
 });
 // Listener 3: inventory update (runs on every emit)
 emitter.on("bookOrdered", (book) => {
 log.push(`Inventory decremented for "${book.title}"`);
 });
 // Listener 4: fires only on the FIRST emit (once)
 emitter.once("bookOrdered", () => {
 log.push("(once-listener) Welcome bonus applied to first order only.");
 });
 // Simulate two orders to show the difference between on() and once()
 emitter.emit("bookOrdered", { title: "Node.js Essentials", price: 299.0 });
 emitter.emit("bookOrdered", { title: "Mastering XML Schemas", price: 450.5 });
 return log;
}
module.exports = demonstrateEvents;

```

**File name : osInfo.js**

**code:**

```

// utils/osInfo.js
// Demonstrates the "os" core module.

const os = require("os");

function getOsInfo() {
 const cpus = os.cpus();
 return {
 platform: os.platform(),
 architecture: os.arch(),
 hostname: os.hostname(),
 cpuModel: cpus.length ? cpus[0].model : "unknown",
 cpuCount: cpus.length,
 };
}

```

```

 totalMemoryMB: (os.totalmem() / (1024 * 1024)).toFixed(2),
 freeMemoryMB: (os.freemem() / (1024 * 1024)).toFixed(2),
 uptimeMinutes: (os.uptime() / 60).toFixed(2),
 homeDir: os.homedir(),
 tempDir: os.tmpdir(),
 nodeVersion: process.version,
 };
}
module.exports = getOsInfo;

```

**File Name :** package.json

**code:**

```

{
 "name": "node-core-modules-server",
 "version": "1.0.0",
 "description": "Week-9 Lab: custom Node.js server with http, os, path, and events core modules",
 "main": "server.js",
 "scripts": {
 "start": "node server.js"
 },
 "license": "MIT"
}

```

**File Name:** pathInfo.js

**code:**

```

// utils/pathInfo.js
// Demonstrates the "path" core module.

const path = require("path");

function getPathInfo(sampleInputPath) {
 const inputPath =
 sampleInputPath || path.join(__dirname, "..", "data", "books.json");

 return {
 inputPath,
 dirname: path.dirname(inputPath),
 };
}

```

```

 basename: path.basename(inputPath),
 extname: path.extname(inputPath),
 parsed: path.parse(inputPath),
 isAbsolute: path.isAbsolute(inputPath),
 normalized: path.normalize(inputPath),
 joinedExample: path.join("data", "books", "bookstore.json"),
 resolvedExample: path.resolve("data", "books", "bookstore.json"),
 platformSeparator: path.sep,
 };
}
module.exports = getPathInfo;

```

**File Name:** server.js

**Code:**

```

// server.js
// Custom Node.js server built with the core "http" module (no Express),
// plus exploration of "os", "path", and "events" core modules.

const http = require("http");
const fs = require("fs");
const path = require("path");
const url = require("url");

const logger = require("./logger");
const getOsInfo = require("./osInfo");
const getPathInfo = require("./pathInfo");
const demonstrateEvents = require("./orderEvents");

const PORT = process.env.PORT || 3000;
const BOOKS_FILE = path.join(__dirname, "books.json");

function sendJSON(res, statusCode, data) {
 res.writeHead(statusCode, { "Content-Type": "application/json" });
 res.end(JSON.stringify(data, null, 2));
}

```

```
function sendHTML(res, statusCode, html) {
 res.writeHead(statusCode, { "Content-Type": "text/html" });
 res.end(html);
}
```

```
function homePage() {
 return `<!DOCTYPE html>
<html>
<head><title>Node.js Core Modules Demo</title>
<style>
 body{font-family:Arial,sans-serif;max-width:640px;margin:40px auto;padding:0 15px;}
 li{margin:8px 0;}
 code{background:#f0f0f0;padding:2px 5px;border-radius:3px;}
</style>
</head>
<body>
 <h1>Custom Node.js Server</h1>
 <p>Built directly on the core <code>http</code> module (no Express),
 with routes demonstrating <code>os</code>, <code>path</code>, and
 <code>events</code>.</p>

 /os — system info via the <code>os</code> module
 /path — <code>path</code> module parsing on the default
books.json path
 /path?file=... — parse a
custom path from a query string
 /events — <code>EventEmitter</code> demo (multiple listeners
+ once())
 /books — JSON read from <code>data/books.json</code> via
<code>fs</code>

 <p>Every request is also logged server-side through a custom
 <code>EventEmitter</code> in <code>utils/logger.js</code> —
 check the terminal running this server.</p>
</body>
</html>`;
}
```

```
const server = http.createServer((req, res) => {
 const parsedUrl = url.parse(req.url, true);
 const pathname = parsedUrl.pathname;

 // Every request emits a 'request' event -> handled in utils/logger.js
 logger.emit("request", {
 method: req.method,
 url: req.url,
 time: new Date().toISOString(),
 });

 if (pathname === "/" && req.method === "GET") {
 sendHTML(res, 200, homePage());
 return;
 }

 if (pathname === "/os" && req.method === "GET") {
 sendJSON(res, 200, getOsInfo());
 return;
 }

 if (pathname === "/path" && req.method === "GET") {
 const sample = parsedUrl.query.file; // optional ?file=... query param
 sendJSON(res, 200, getPathInfo(sample));
 return;
 }

 if (pathname === "/events" && req.method === "GET") {
 const log = demonstrateEvents();
 sendJSON(res, 200, { description: "EventEmitter demo output", log });
 return;
 }

 if (pathname === "/books" && req.method === "GET") {
 fs.readFile(BOOKS_FILE, "utf8", (err, data) => {
```

```
if (err) {
 sendJSON(res, 500, {
 error: "Could not read books.json",
 details: err.message,
 });
 return;
}
sendJSON(res, 200, JSON.parse(data));
});
return;
}
```

```
sendHTML(
 res,
 404,
 `

404 Not Found

No route for ${pathname}

`
\);
}\);
```

```
server.listen(PORT, () => {
 console.log(`Server running at http://localhost:${PORT}/`);
});
```